Documento de Diseño del Sistema - Versión Ampliada

Carpeta 4: Arquitectura, Estructura del Proyecto y Diseño UI/UX

Autor: Yerson Stiven Cuellar Rubiano | **Programa:** Análisis y Desarrollo de Software (ADSO) | **Institución:** SENA Centro de la Industria (Neiva, Huila) | **Versión:** 2.0 (Detallada y Unificada)

1. Introducción y Propósito del Diseño

El presente documento abarca la totalidad de la fase de **Diseño (Carpeta 4)**. Su propósito es traducir los requisitos funcionales y analíticos definidos en las fases anteriores en planos técnicos concretos. En este entregable se define la arquitectura de software, la estructuración de los repositorios de código, la selección justificada de tecnologías y las directrices del diseño de interfaces (UI) y experiencia de usuario (UX). Este documento sirve como mapa de ruta definitivo antes de iniciar la etapa de codificación pura.

2. Arquitectura del Sistema (system-architecture)

El proyecto se aleja de los modelos monolíticos tradicionales y adopta una **Arquitectura Cliente-Servidor Desacoplada y Orientada a Servicios (SOA)**. Esta separación garantiza que el fallo de un componente no comprometa la integridad de toda la plataforma.

Modelo de Interacción por Capas

- **Capa de Presentación (Frontend):** Dividida en dos sub-aplicaciones. El Panel Administrativo Web, que se procesará mediante Server-Side Rendering (SSR) para garantizar seguridad en las rutas protegidas, y la Aplicación Móvil, que se ejecutará nativamente del lado del cliente (Client-Side).
- **Capa de Lógica de Negocio (Backend y APIs):** Supabase actuará como el núcleo central, exponiendo automáticamente una API RESTful y GraphQL segura. Las reglas del negocio (como impedir que un cliente pague algo con precio modificado) residirán en esta capa.
- **Capa de Persistencia de Datos:** Una base de datos PostgreSQL gestionada, con almacenamiento en la nube adicional para los recursos multimedia (imágenes del catálogo).
- **Capa de Servicios Externos (Third-Party):** Conexiones HTTPS hacia Wompi para la tokenización de pagos y WebSockets hacia Google Maps para el servicio de geocodificación espacial.

3. Tecnologías Utilizadas y Justificación (technologies-used)

La selección del stack tecnológico se realizó priorizando la velocidad de desarrollo, el rendimiento en producción y la mantenibilidad a largo plazo.

Tecnología	Capa de Aplicación	Justificación Técnica de Selección
Next.js (v14+)	Frontend Web	Framework de React superior para aplicaciones empresariales. Su sistema de enrutamiento (App Router) y renderizado en el servidor protege los paneles administrativos y reduce el tiempo de carga visual inicial a menos de 1 segundo.
React Native & Expo	Frontend Móvil	Permite compilar código JavaScript/TypeScript en binarios nativos para Android y iOS. Expo simplifica el acceso al hardware del teléfono (GPS, Notificaciones Push, Cámara) sin necesidad de escribir código en Java o Swift.
TypeScript	Lenguaje Base	Añade tipado estático a JavaScript. Es obligatorio en este proyecto para modelar interfaces de datos (ej. `IPedido`), previniendo errores de variables nulas o tipos incorrectos antes de ejecutar la aplicación.
Supabase	Backend & Auth	Provee una base de datos relacional PostgreSQL con políticas de seguridad integradas (RLS) y un sistema de autenticación JWT robusto listo para producción, eliminando meses de desarrollo backend.
Tailwind CSS + shadcn/ui	Estilos y UI	Tailwind permite estilizar componentes mediante clases de utilidad directamente en el código, evitando archivos CSS gigantes. shadcn/ui provee componentes accesibles (botones, modales, tablas) con un diseño corporativo moderno.
Zustand	Gestión de Estado	Librería ultraligera para el manejo del estado global de la aplicación (ej. mantener los datos del carrito de compras). Supera a Redux en simplicidad y previene re-renderizados innecesarios.

4. Estructura del Proyecto (project-structure)

El código fuente se organizará bajo una metodología de separación por dominios. Esto facilita la lectura del código, el mantenimiento y futuras auditorías. A continuación, se presenta el esquema oficial del árbol de directorios:

```
📦 PROYECTO_DELIVERY_SENA
 ├ 📂 admin-web/              (Proyecto Next.js - Panel de Restaurante)
 │ ├ 📂 src/
 │ │ ├ 📂 app/                (Rutas protegidas: /dashboard, /pedidos, /menu)
 │ │ ├ 📂 components/         (Componentes UI reutilizables de shadcn)
 │ │ ├ 📂 lib/                (Clientes de Supabase, formateadores de moneda)
 │ │ ├ 📂 types/              (Definiciones de TypeScript globales)
 │ │ └ 📂 hooks/              (Custom hooks para llamadas a BD)
 │ └ 📜 tailwind.config.ts    (Variables de diseño y colores corporativos)
 │
 ├ 📂 mobile-app/            (Proyecto React Native - App Cliente/Repartidor)
 │ ├ 📂 src/
 │ │ ├ 📂 screens/            (Vistas completas: Home, Cart, Checkout, Map)
 │ │ ├ 📂 navigation/         (Navegación Stack y Tabs mediante React Navigation)
 │ │ ├ 📂 services/           (Funciones que consumen la API de Wompi y Maps)
 │ │ ├ 📂 store/              (Estado global Zustand para el Carrito)
 │ │ └ 📂 assets/             (Imágenes estáticas, fuentes, iconos)
 │ └ 📜 app.json              (Configuración de Expo y permisos de GPS/Push)
 │
 └ 📂 database/              (Scripts SQL)
   └ 📜 schema.sql           (Creación de tablas, triggers y políticas RLS)
```

5. Diseño de Interfaces (UI) y Experiencia de Usuario (UX) (ui-ux-design)

El enfoque gráfico y funcional de la aplicación se basará en principios de "Mobile-First" y diseño minimalista, reduciendo la fricción cognitiva del usuario durante el proceso de compra.

A. Sistema de Diseño (Design System)

- **Tipografía:** Se utilizará la familia *Inter* para entornos web y *Roboto/San Francisco* para aplicaciones nativas. Estas fuentes Sans-Serif garantizan legibilidad óptima en descripciones largas de productos.
- **Esquema de Colores (Brand Guidelines):**
  - `Primario (#2563eb - Azul Rey):` Utilizado exclusivamente para llamadas a la acción críticas (Botón "Pagar", "Agregar al Carrito").
  - `Fondo General (#f1f5f9 - Gris Claro):` Para separar visualmente las tarjetas de productos sin saturar la pantalla.
  - `Éxito (#16a34a - Verde):` Para confirmaciones de pago y estados "Entregado".
  - `Alerta (#dc2626 - Rojo):` Para cancelación de órdenes o errores bancarios.

B. Flujos de Experiencia de Usuario (UX)

El diseño prioriza la conversión de ventas. El carrito de compras será siempre visible (Floating Action Button o Tab inferior). El proceso de pago constará de máximo tres toques (Three-Tap Checkout) y la pasarela Wompi se integrará como un WebView in-app para evitar que el usuario deba cambiar de aplicación (lo cual suele provocar caídas en las ventas).

C. Prototipado de Alta Fidelidad (prototypes.fig)

Antes del desarrollo, todas las pantallas han sido vectorizadas e interconectadas en **Figma**. Este lienzo digital interactivo permite al equipo simular clics, transiciones y alertas sin escribir código, validando el recorrido del cliente. *(El enlace al lienzo interactivo de Figma se anexa como entregable digital para la validación de los instructores).*